

Raport architektoniczny (M4) — 10xCardsAstro

Autor: Michał Dudziak · **Repozytorium:** <https://github.com/dudziakm/10xCardsAstro> **Data:** 5 sierpnia 2026 ·
Weryfikowane wobec: master @ 0d1f5ca

Raport syntetyzuje cztery artefakty Modułu 4: mapę repozytorium (L2), research wybranego ficzera (L3), plan refaktoryzacji (L4) i notatki domenowe inspirowane DDD (L5). Każdą liczbę poniżej odtworzyłem na master 5 sierpnia — sekcja „Granice pewności” mówi wprost, co jest wykonane, a co pozostaje planem. Rozróżnienie jest celowe: to aplikacja w większości napisana przez agenta, więc dokument planistyczny i kod mogą powtarzać to samo założenie, zamiast niezależnie się potwierdzać.

1. Mapa repozytorium (L2)

Ryzyko skupia się w jednym przepływie: /learn → LearningSession → GET/POST /api/learn/session → LearningService → ReviewScheduler + Supabase. Churn na pełnej historii wskazuje te same pliki, co analiza ryzyka: package.json (19 zmian), e2e/05-learning-session.spec.ts (15), .github/workflows/ci.yml (14), src/pages/api/learn/session.ts (13), src/middleware/index.ts (11). Historia jest nietypowa — 92 commity w czerwcu 2025, potem dziesięć miesięcy ciszy, a wznowienie w 2026 to niemal wyłącznie bumpy zależności. Autorstwo: 81 commitów agenta, 30 jedynego człowieka, 8 dependabota.

dependency-cruiser 18.1.0 przechodzi 65 modułów i 104 zależności bez cyklu i bez importów API ↔ React islands. Artefakt sam zapisuje, czego to **nie** dowodzi: graf importów nie widzi runtime fetch w LearningSession, plików .astro ani polityk bazy. Weryfikacja AST domyka to, czego graf nie pokazuje — w produkcji istnieją dokładnie dwa miejsca konstrukcji LearningService (session.ts:65, rate.ts:23).

→ [context/map/](#)

2. Research ficzera: postęp nauki (L3)

Research przeszedł przepływ z cytatami plik:linia i wyprodukował siedem pozycji długu. Trzy są rdzeniowe:

- Postęp liczony przy prezentacji, nie przy ocenie.** getNextCard inkrementuje cards_reviewed (learning.service.ts:91-100) i zwraca już podbitą wartość; rateFlashcard nie inkrementuje jej wcale. Odświeżenie GET zawyża statystykę.
- Selekcja due po limit(10).** Zapytanie sortuje po created_at i bierze ≤10 rekordów (:45-60), a filtr next_review_date działa dopiero w pamięci (:62-76). Karta jedenasta może nigdy nie wrócić. Artefakt zapisuje to jako „inferencja logiczna... nie obserwacja produkcyjna”.
- RLS wyłączone przez ostatnią migrację** tabel nauki (20240320140000_disable_learning_rls_for_testing.sql:6-8).

Do tego: GET wykonuje mutacje (inkrement licznika, ?reset=true usuwa postęp), brak niezmiennika „ocenić można tylko kartę przedstawioną”, brak atomowości między upsertem postępu a licznikiem sesji. Tabela luk testowych ma 13 wierszy; jako krytyczną wskazuje brak testu izolacji użytkownik A vs B na tabelach nauki. Research obnaża też pułapkę nazewniczą: session.test.ts nosi nazwę „API integration tests”, ale nie importuje handlerów ani produkcyjnego serwisu — testuje atrapy i obliczenia na danych.

→ [context/changes/learning-progress-analysis/](#)

3. Plan refaktoryzacji (L4)

Trzy kandydatury oceniłem wzorem (impact + frequency + confidence) - (cost + migration risk): przywrócenie RLS 7, przeniesienie selekcji due do zapytania 8, naliczanie postępu po ocenie 6. Plan nazywa powstałe napięcie wprost: ranking ryzyka biznesowego stawia RLS pierwszy, ale ranking punktowy stawia zapytanie — i to nie jest ta sama kolejność co kolejność implementacji.

Rozstrzygnięciem było **nie** implementować żadnej z trzech od razu. Phase 1 wydziela czysty `ReviewScheduler` z testami `characterization`: mały, odwracalny seam, który nie rusza zapytań ani danych. Charakteryzacja 24/24, scalone jako [PR #25](#) (2a2b929) — jest na `master`, nie tylko lokalnie. Phase 2–5 (RLS, zapytanie due, transakcyjne naliczanie, agregat + ACL) pozostają planami; `change.md` zabrania łączenia ich w jeden refaktor przed osobną weryfikacją środowiska Supabase. W momencie pisania planu (31 lipca) baseline był czerwony: 88/101 testów i lint przewracający się na wygenerowanym `database.types.ts`. Dziś ten sam, nietargetowany przebieg jest zielony — remediacja weszła osobno, po Phase 1.

→ [context/changes/harden-learning-progress/](#)

4. Notatki domenowe inspirowane DDD (L5)

Destylacja języka dała dziesięć pojęć ze wskaźnikami do kodu — i jedno odkrycie, które porządkuje cały problem: „**Przedstawienie karty**” (**Card Presentation**) **nie istnieje w kodzie**. Nie jest źle zaimplementowane; jest nieobecne. Ocena sprawdza wyłącznie własność sesji i karty (`learning.service.ts:134–157`). Wszystkie trzy rdzeniowe rozjazdy MODEL vs KOD (z ośmiu w tabeli) wynikają z tej luki albo z wyłączonego RLS.

Stąd niezmiennik rdzeniowy: *rating można zaakceptować dokładnie raz, tylko dla karty należącej do użytkownika, wcześniej przedstawionej w tej samej aktywnej sesji; dopiero zaakceptowany rating zwiększa `cardsReviewed`*. Agregatem priorytetowym jest `LearningSession`, z siedmioma nazwanymi błędami domenowymi zmapowanymi na HTTP (`CardNotPresented` → 409, `PresentationAlreadyRated` → 409, `SessionOwnershipMismatch` → 404/403 ...) i siedmiofazową ścieżką forward-only. Plan ACL zawęża wyciek SDK do slice'u nauki — porty `LearningReadRepository` / `LearningWriteRepository`, adapter `SupabaseLearningRepository` — i jawnie **nie** proponuje wrappera całego SDK. RLS zapisałem jako finding priorytetowy, ale kontrolę Generic/security, nie kandydata na agregat.

→ [context/domain/](#)

Granice pewności (weryfikacja 5 sierpnia 2026)

Sprawdzone na `master @ 0d1f5ca`: pełna suite 101/101 (vitest, 10 plików); graf zależności 65 modułów / 104 zależności, zero naruszeń; `npm audit --omit=dev` → 0 podatności; najnowszy zielony CI [run 31020657326](#).

Niesprawdzone i celowo nie twierdzone: zdalna konfiguracja RLS i ledger migracji w Supabase, dane produkcyjne, intencjonalna semantyka losowości w wyborze karty, publiczny hosting. Agregat, ACL, transakcja i RLS **nie są wdrożone** — to plany z mierzalnymi kryteriami odbioru.

Otwarte operacyjnie: rotacja klucza dostawcy AI, który wyciekł do diffu nieudanego testu w trakcie Phase 1 (incydent zapisany w `plan.md`; klucz nie znajduje się w plikach ani diffie tej zmiany).

Wniosek

Repozytorium ma mapę, ranking oparty na jawnym wzorze i jeden bezpieczny seam scalony do `master`. Kolejność dalszych kroków wynika z ryzyka, nie z wygody: najpierw RLS na tabelach nauki wraz z testem izolacji A/B, potem przeniesienie selekcji due do zapytania z fixture na karcie jedenastej, na końcu przeniesienie licznika do transakcji przy ocenie. Każdy z tych kroków wymaga własnej weryfikacji środowiska — dlatego żaden nie wszedł razem z Phase 1.